Interim Report

The Automaton Auditor

Week 2 Challenge

Martha Ketsela Mengistu

February, 2026

1. Architecture Decision

The **Automaton Auditor** is a deep multi-agent governance system built on a hierarchical LangGraph StateGraph. It follows a "Digital Courtroom" model using a three-layer orchestration.

The State Flow Architecture

This architecture is designed to move from Forensic Evidence Collection to Dialectical Synthesis.

Layer 1: Forensic Detectives (Parallel Fan-Out)

- **RepoInvestigator**: Performs AST-based analysis of the codebase and extracts git history.
- **DocAnalyst**: Ingests the PDF report and cross-references claims against actual file paths.
- **VisionInspector**: Analyzes architectural diagrams to verify they match the actual code structure.
- **State Reduction**: These nodes run concurrently and save their findings into a shared evidences dictionary using the operator.ior reducer to prevent data overwriting.

Layer 2: The Synchronization Point (Fan-In)

- **EvidenceAggregator:** This node waits for all three detectives to complete their tasks.
- **Function:** It synthesizes the raw data into a structured format that the Judicial Layer can interpret.

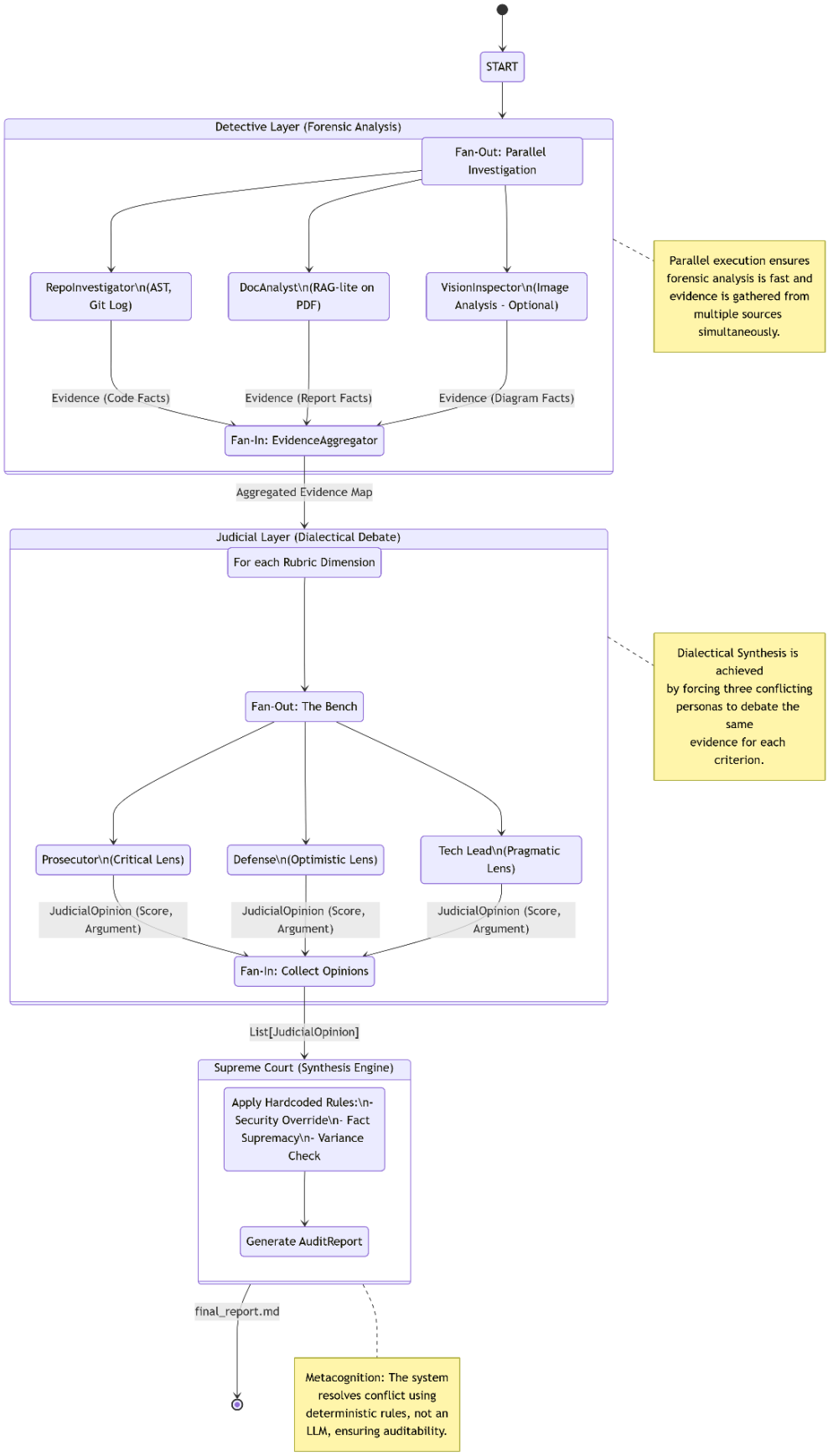
Layer 3: The Judicial Bench (Parallel Fan-Out)

Once evidence is gathered, the graph triggers three parallel nodes with distinct, conflicting personas:

- **The Prosecutor**: Scrutinizes evidence for gaps and security flaws.
- **The Defense**: Highlights effort, intent, and creative problem-solving.
- **The Tech Lead**: Evaluates architectural soundness and technical debt.
- **Constraint**: Each judge must use structured output (Pydantic) to ensure the opinions list is populated correctly via the operator.add reducer.

Layer 4: The Supreme Court (Final Synthesis)

- **ChiefJusticeNode**: This is the final node that applies hardcoded deterministic rules, such as the Rule of Security (capping scores for vulnerabilities) and the Rule of Evidence (overruling claims not backed by the RepoInvestigator).
- **Output**: The node generates a finalized Markdown Audit Report containing the executive summary and remediation plan.


START
Detective Layer (Forensic Analysis)
Fan-Out: Parallel Investigation
RepoInvestigator\n(AST, Git Log)
DocAnalyst\n(RAG-lite on PDF)
VisionInspector\n(Image Analysis - Optional)
Parallel execution ensures forensic analysis is fast and evidence is gathered from multiple sources simultaneously.
Evidence (Code Facts)
Evidence (Report Facts)
Evidence (Diagram Facts)
Fan-In: EvidenceAggregator
Aggregated Evidence Map
Judicial Layer (Dialectical Debate)
For each Rubric Dimension
Dialectical Synthesis is achieved by forcing three conflicting personas to debate the same evidence for each criterion.
Fan-Out: The Bench
Prosecutor\n(Critical Lens)
Defense\n(Optimistic Lens)
Tech Lead\n(Pragmatic Lens)
JudicialOpinion (Score, Argument)
JudicialOpinion (Score, Argument)
JudicialOpinion (Score, Argument)
Fan-In: Collect Opinions
List[JudicialOpinion]
Supreme Court (Synthesis Engine)
Apply Hardcoded Rules:\n- Security Override\n- Fact Supremacy\n- Variance Check
Generate AuditReport
final_report.md
Metacognition: The system resolves conflict using deterministic rules, not an LLM, ensuring auditability.

1.1. Why Pydantic over Plain Dicts for State

Decision: Use Pydantic BaseModel for structured data (Evidence, JudicialOpinion) and a TypedDict with Annotated fields for the main AgentState.

Failure Mode Prevented (Dict Soup & Data Corruption): Using plain Python dictionaries for complex state is brittle. In a parallel graph execution, multiple nodes writing to the same dict key can lead to silent data overwrites or "hallucination" where one node's output is lost. This violates the principle of forensic accuracy.

Alternatives Considered: Plain dicts, dataclasses.

Why Pydantic/TypedDict?

1. **Structured Validation:** Pydantic enforces that an Evidence object must have a goal, found bool, and location. This prevents a detective node from returning malformed evidence that could crash downstream judges.
2. **Immutable Reducers (The Decisive Factor):** TypedDict combined with Annotated[..., operator.add] for the opinions list is the only way to guarantee that when the Prosecutor, Defense, and Tech Lead run in parallel, their outputs are appended to the state, not overwritten. This is critical for the dialectical process.
3. **Self-Documenting:** The AgentState definition acts as a single source of truth for what data flows through the graph, making the system more maintainable and less prone to "spaghetti data" anti-patterns.

1.2. How AST Parsing Was Structured (and Why Not Regex)

Decision: Use Python's ast module to analyze code structure within repo_tools.py.

Failure Mode Prevented (Brittle Analysis): Relying on regex to find StateGraph patterns is a classic "vibe coding" failure mode. A simple regex for add_edge would break on multi-line function calls, different spacing, or commented-out code. It would also flag imports as usage. This leads to false positives and an unreliable audit.

Alternatives Considered: Regex, simple string searching (in operator), Tree-sitter.

Why AST?

1. **Structural Understanding:** AST parsing allows us to distinguish between import langgraph and builder.add_edge('node_a', 'node_b'). We can verify not just the presence

of a string, but the instantiation and method calls on the graph object. This provides "irrefutable and precise" evidence as required by the rubric.

2. **Handling Complexity:** The ast module reliably handles nested structures, multi-line function calls, and complex Python syntax that regex cannot.
3. **Why not Tree-sitter?** While more language-agnostic, Tree-sitter adds an external dependency and complexity. Since our target is a Python codebase, the built-in ast module is the most lightweight and Pythonic way to achieve deep forensic analysis.

1.3. Sandboxing Strategy for Cloning Unknown Repos

Decision: Use tempfile.TemporaryDirectory to create an ephemeral, isolated environment for all git operations.

Failure Mode Prevented (Security Negligence & Side Effects): Cloning arbitrary code from the internet is a massive security risk. A repository could contain malicious scripts in its hooks or be structured to path-traverse out of the clone directory if not handled properly. Running os.system('git clone ' + repo_url) directly in the working directory is a critical security violation.

Alternatives Considered: Cloning to a fixed ./temp_repos directory, using Docker.

Why tempfile?

Guaranteed Isolation: TemporaryDirectory ensures the repository is cloned to a unique, system-created location. When the with block exits, the directory and all its contents are automatically and completely destroyed. This prevents disk space leaks and ensures one audit cannot interfere with another.

2. Gap Analysis and Plan

Completed

- **State Definitions (src/state.py)**: Pydantic models for Evidence, JudicialOpinion, AuditReport and a TypedDict for AgentState with reducers are fully implemented.
- **Core Forensic Tools (src/tools/)**: repo_tools.py implements AST-based graph analysis and sandboxed git history extraction. doc_tools.py implements a RAG-lite PDF ingestion system using Docling for text extraction.
- **Detective Nodes (src/nodes/detectives.py)**: RepoInvestigatorNode and DocAnalystNode are implemented as LangGraph nodes that output structured lists of Evidence.
- **Graph Wiring (src/graph.py - Partial)**: The graph successfully implements the detective layer fan-out/fan-in, culminating in an EvidenceAggregator node.

Judicial Layer

If the prompts for the judges are too similar (e.g., generic "grade this"), they will produce convergent scores. This collapses the dialectical tension, making the synthesis step trivial and robbing the final verdict of its nuanced "dissent."

Actionable Plan:

1. **Prompt Engineering (High Specificity):** I will craft system prompts with explicit, conflicting philosophies drawn from the challenge spec.
    - **Prosecutor Prompt:** "You are a cynical, detail-oriented prosecutor. Your job is to assume the developer cut corners. Scrutinize the Evidence for absence. If a file exists but is incomplete, argue for a low score. Your core question: 'What is missing?'."
    - **Defense Prompt:** "You are an empathetic defense attorney. Your job is to assume good intent and look for the 'spirit of the law.' Highlight clever workarounds and effort. If a component is partially implemented, argue for partial credit. Your core question: 'What was attempted, and what can be learned from it?'."
    - **Tech Lead Prompt:** "You are a pragmatic Tech Lead. You don't care about intent or cynicism. You only care about the artifacts. Is the code maintainable? Does the architecture actually solve the problem? If the AST parsing is deep but the graph doesn't compile, you must score based on the broken artifact. You are the tie-breaker."

2. **Graph Modification:** Define the nodes for Prosecutor, Defense, and TechLead judges.py. Modify the src/graph.py to add a second fan-out node that, for each criterion in the rubric, invokes the three judge nodes in parallel.

Synthesis Engine

Using an LLM to synthesize the judges' opinions is just "averaging with extra steps." It loses the determinism and auditability required for a production system.

Actionable Plan:

1. **Deterministic Python Logi**c: The ChiefJusticeNode will be implemented with pure Python logic, not an LLM call. It will ingest the list of JudicialOpinion objects for a criterion.

2. **Hardcoded Rule Application**: The node will execute a sequence of rules based on the synthesis_rules from the JSON constitution:
    - **Fact Supremacy:** Check if any Defense opinion cites evidence that contradicts the Evidence objects (e.g., claims a file exists when found=False). If so, overrule the Defense and log the dissent.

- **Security Override:** Scan all opinions and cited evidence for confirmed security flaws (e.g., raw os.system). If found, set a flag to cap the final score for the relevant criterion at 3.
- **Variance Re-evaluation:** If the variance of the three scores > 2, log a dissent_summary explaining the core conflict (e.g., "Prosecutor cites missing file X, Defense argues for effort based on complex AST logic."). The final score will be the median, but the dissent is explicitly captured.

3. **Markdown Generation:** The final step will serialize the AuditReport Pydantic model into a professional Markdown file, ensuring the output is usable and meets the "Executive Grade" standard.